



Rishabh Singh
Roll-no.23f2000701

MAD-1

Project

Report

Quiz-Master-v1

Project Report: Quiz-Master-v1

1) Student details

Name: Rishabh Singh

Roll-number: 23f2000701

Term of project: January 2025

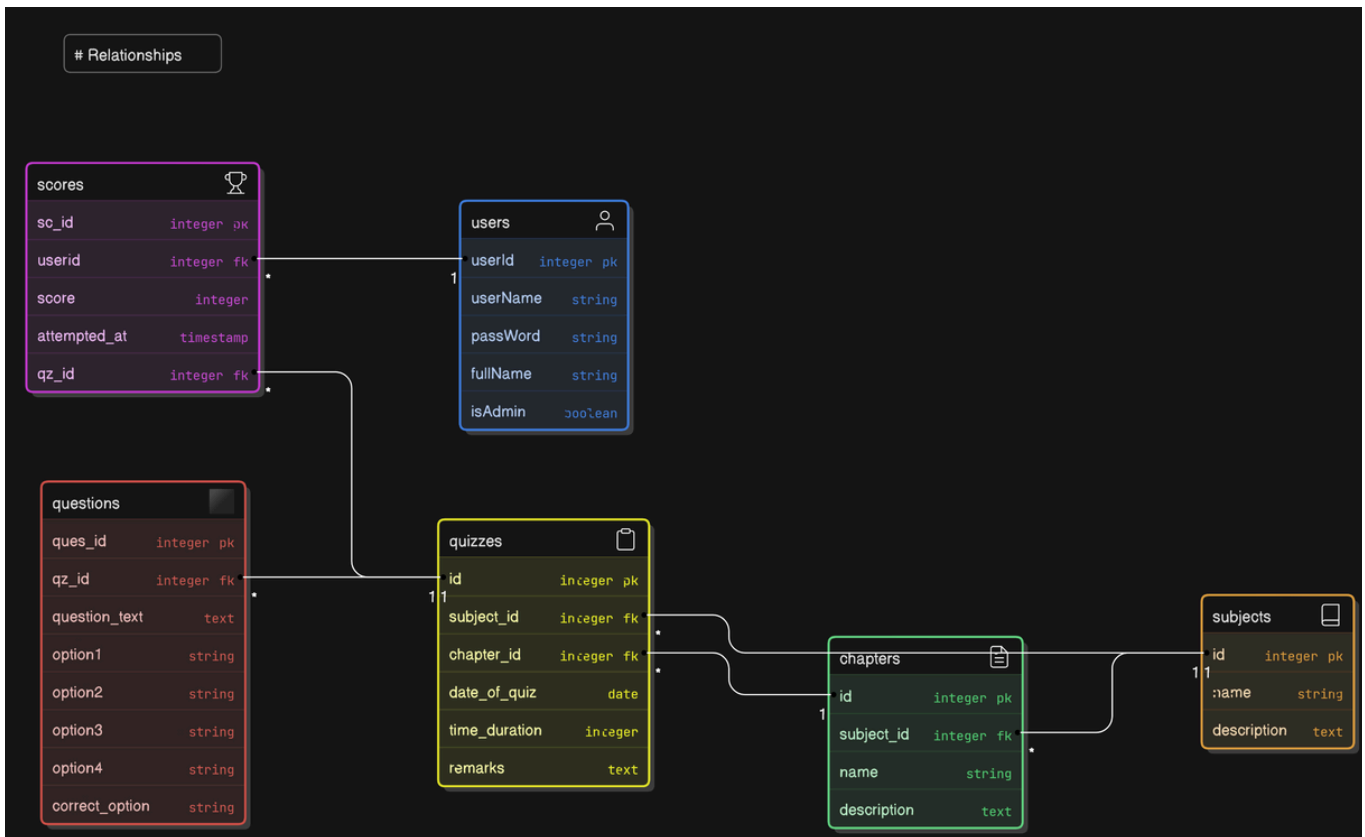
2) Project description:

Quiz-Master-v1 is a web application designed to facilitate learning through quizzes. It allows an administrator to create quizzes across various subjects and chapters, add questions to these quizzes, and subsequently enable students to take these quizzes to evaluate their understanding of specific topics. This interactive platform not only helps students test their knowledge but also provides administrators with tools to monitor student performance and engagement.

3)Technology Stack:

- **Flask** : A lightweight Python web framework
- **Jinja2** : A templating engine for generating HTML
- **SQLite** : Database management system for storing application data.
- **SQLAlchemy**:ORM(Object-Relational-Mapping) tool for database interaction
- **Datetime** : Python library for handling date and time operations
- **HTML**: Markup language for creating web pages.
- **CSS**: Stylesheet language for designing web pages.
- **JavaScript**: Programming language for interactive webcontent.
- **ChartJS**: Library for creating interactive charts.

4) Database Schema:



The database schema consists of six tables:

- **Users:** Stores user details with `userid` as the primary key. It includes `userName`, `passWord`, `fullName`, and `isAdmin` to indicate admin status.
- **Subjects:** Contains subject details with `id` as the primary key, including `name` and `description`.
- **Chapters:** Represents chapters within subjects, with `id` as the primary key and `subject_id` as a foreign key referencing `Subjects`.
- **Quizzes:** Stores quiz details with `id` as the primary key. It links to `Subjects` and `Chapters` via `subject_id` and `chapter_id`. Includes `date_of_quiz`, `time_duration`, and optional `remarks`.
- **Questions:** Holds quiz questions with `ques_id` as the primary key. It references `quizzes` via `qz_id` and includes the question text, four options, and the correct answer.
- **Scores:** Stores quiz scores with `sc_id` as the primary key. It references `Users` (`userid`) and `Quizzes` (`qz_id`). Also records the `score` and `attempted_at` timestamp.

5)Architecture

- **controllers/**: Contains Python files handling the application's logic, such as processing requests and interacting with models.
- **models/**: Defines the database schema and ORM models.
 - **models.py**: Contains SQLAlchemy model definitions for the database tables.
 - **db.py**: Initializes the SQLAlchemy database instance.
 - **create_db.py**: Script to create the database and tables based on the defined models.
- **static/**: Stores static assets like CSS code, and images for the frontend.
- **templates/**: Contains HTML templates using Jinja2 for rendering dynamic web pages.
- **app.py**: The main entry point of the Flask application, initializing routes, database connections, and configurations.
- **README.md**: Provides an overview of the project, including setup instructions and usage details.

6)Features

The Quiz Master web application provides an interactive platform for exam preparation, supporting multiple courses and structured quizzes.

User Roles:

- Admin (Quiz Master): Has complete control over users and quiz management. Can create subjects, chapters, quizzes, and questions.
- Users: Can register, log in, attempt quizzes, and view past scores.

Quiz Management:

- Admin can create, edit, and delete subjects, chapters, quizzes
- Admin can add multiple-choice questions (MCQs) to quizzes.

User Experience:

- Users can browse available quizzes
- Users can start quizzes with a preset duration.
- Quiz scores are stored, and previous scores can be reviewed.

Admin Dashboard:

- Search functionality for searching users, subjects, and quizzes.
- Summary charts for quiz performance insights.

User Dashboard:

- Displays available quizzes and past performance.
- A quiz can be attempted any number of times.

7) Important API endpoints:

Create (POST)

1. POST/register – Register a new user.
2. POST/admin/subjects/new – Create a new subject.
3. POST/admin/subjects/<int:subject_id>/details/<int:qz_id>/add_questions – Add questions to a quiz.
4. POST/admin/subjects/<int:subject_id>/<int:chapter_id>/details/create – Create a new quiz.

Read (GET)

1. GET/admin – View the admin dashboard.
2. GET/student/scores – View student scores.
3. GET/student/performance – View student performance summary.
4. GET/admin/subject/<int:subject_id>/chapters_list – View chapters of a subject.
5. GET/student/view_quiz_details/<int:quiz_id> – View quiz details before attempting.

Delete (POST)

1. POST/admin/subjects/delete/<int:subject_id> – Delete a subject.
2. POST/admin/subjects/<int:subject_id>/chapters/<int:chapter_id>/quiz/<int:quiz_id>/delete – Delete a quiz.
3. POST/admin/subjects/<int:subject_id>/details/<int:qz_id>/delete_question/<int:ques_id> – Delete a question.
4. POST/admin/subjects/delete_chapter/<int:chapter_id> – Delete a chapter.

8) Video Demo link:

Google drive link of Project demonstration video:

<https://drive.google.com/file/d/1fyREXVK-9kGObkzLPkBX03ek7fyemMuE/view?usp=sharing>

THANK YOU